ENGR 102 Summer Bridge

Day 11 Instructor Key

While Loops, Update Order, Sentinels, and Termination

Monday, July 20, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Design loops whose state changes make termination demonstrable.

Allowed tools	Prior tools plus while loops and sentinel-controlled input
Support supplied	The sentinel, valid-data interval, and required summary are supplied.
You must decide	Loop pattern, update placement, validity branch, and the empty-data case.
Scaffold level	Termination evidence prompted; algorithm no longer sequenced line by line.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	12	intermediate state, condition results, and exact output
Diagnosis and repair	10	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	28	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 12 points

Trace both parts of the compound condition before each pass and the state after each update.

```
level = 17
steps = 0

while level < 50 and steps < 4:
    level = level + 9
    steps = steps + 1
    print(level, steps)

print("final", level, steps)
```

Your task

1. Build one ledger with condition result, level, and steps before and after each pass.
2. Give every output line and explain which condition evidence proves termination.

Answer and explanation

1. Passes begin at (17,0), (26,1), (35,2), and (44,3), with both comparisons True each time. After updates the states are (26,1), (35,2), (44,3), and (53,4).
2. The loop prints 26 1; 35 2; 44 3; 53 4; then final 53 4. The next condition is False because level < 50 is False and steps < 4 is also False. Either false operand is enough to stop an **and** condition.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 10 points

This loop should read integers until -1 and count nonnegative entries, but it can repeat forever.

```
reading = int(input())
valid_count = 0

while reading != -1:
    if reading >= 0:
        valid_count = valid_count + 1

print(valid_count)
```

Your task

1. Explain the infinite-loop condition using the unchanged state, not just the phrase missing update.
2. Write the minimal repair and describe the final false condition for inputs 5, 8, -1.

Answer and explanation

1. If the first reading is not -1, reading never changes in the body. Therefore reading != -1 remains True forever, and the same value is repeatedly counted when it is nonnegative.
2. Add reading = int(input()) as the final statement in the loop body. For inputs 5, 8, -1, valid_count becomes 2; after -1 is stored, reading != -1 is False and the loop stops before counting the sentinel.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 28 points across Pages 4-5

Program specification

- Read integer readings until the sentinel -1 is entered.
- Values from 0 through 100 inclusive are valid. Any other non-sentinel value is ignored and must print Ignored.
- For valid values, compute count, total, and the number at or above 70.
- After termination, print count, total, and high_count on separate lines. Then print the average, or print No data when count is zero.

Required planning evidence

1. State the loop-controlling variable and show where it changes so that every path either terminates or reaches another input operation.

Answer and explanation

```
reading = int(input())
count = 0
total = 0
high_count = 0

while reading != -1:
    if 0 <= reading <= 100:
        count = count + 1
        total = total + reading
        if reading >= 70:
            high_count = high_count + 1
    else:
        print("Ignored")
    reading = int(input())

print(count)
print(total)
print(high_count)
if count == 0:
    print("No data")
else:
    print(total / count)
```

The first input occurs before the condition, and exactly one new input occurs at the bottom of every completed pass. This guarantees progress toward a future sentinel regardless of whether a value is valid.

The average branch is after the loop. It protects division by zero while preserving all requested summary output.

Task 3 continued. Test and defend the program **included in 28 points**

Expected tests and results

1. Inputs 70, 101, 20, -1 print Ignored during processing, then 2, 90, 1, and 45.0 on separate summary lines.
2. Immediate -1 prints 0, 0, 0, and No data.

Scoring guidance

3 points termination plan; 4 initialization/input; 6 sentinel loop and update; 5 validity handling; 4 summaries; 3 empty case; 3 tests/explanation.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 11 • Contract 16b4e542994c

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 5 **Activities:** 18.19, 18.21, 18.22

Complete the same while-loop cells with less planning support and defend the empty or already-finished case.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.19 18-19-work / 18-19-transfer / 18-19-cold	**Countdown To Multiple**	Write a while condition from a divisibility goal.
18.21 18-21-work / 18-21-transfer / 18-21-cold	**Cooldown Sequence**	Track a current value and a stored sequence.
18.22 18-22-work / 18-22-transfer / 18-22-cold	**Rounds To Clear Queue**	Model repeated queue reduction.

Readiness check before you code

1. Predict whether each loop executes zero, one, or multiple times for its transfer case before running it.

   Answer and explanation: Predictions must follow the actual transfer inputs and initial condition. Credit the reasoning even if the initial prediction is later revised from output evidence.

2. Identify where output or list append must occur relative to the update.

   Answer and explanation: 18.21 appends the updated state so the final nonpositive value is retained; 18.22 increments once per completed reduction round; 18.19 prints only after the first matching value is reached.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those studio cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.